

contexture

User manual

Contents

1. What this package is for	2
2. What else you could build	2
3. Installation and compiling	2
4. The anchor primitive: <code>anchor</code> , <code>anchors</code>	3
4.1. Re-emitting stored content: <code>strip-labels</code>	4
4.2. A fuller example: list of figures	6
5. <code>satellite</code> and <code>bundle</code> : the shared pilot	8
6. Two independent compile axes: <code>variant</code> , <code>preview</code>	8
7. Diagnostics: <code>diagnose</code> , <code>set-strict</code>	9
8. <code>xref</code> : cross-references with a real page number	10
9. Composing independent packages	10

1. What this package is for

Typst’s experimental bundle export (`--features bundle --format bundle`) lets one compile produce several named documents that share one introspection space: a `query()` run from any one of them sees content laid out in **all** of them, with real, final page numbers — because they were genuinely composed together, in the same pass. `contexture` is the backend that turns that raw capability into something pleasant to build on: one primitive to anchor a piece of data anywhere in the bundle and resolve it from anywhere else (`anchor/anchors`), one function that owns the only `document(...)` call in the whole bundle so independently-written packages never fight over it (`bundle/satellite`), two small compile-time flags shared by every package built on it (`variant/preview`), and a common way to report something wrong without necessarily failing the build (`diagnose/set-strict`).

`contexture` itself knows nothing about revisions, checklists, glossaries, or any other specific document type — it only knows how to anchor data, resolve it across documents, and hand out real files at the end. Every example in the next few chapters builds something new directly on these primitives, with no other package involved at all, to show the mechanism stands on its own two feet. Only in the last chapter, “Composing independent packages,” does a **pair** of real packages built on `contexture` — `@preview/palimpsest` and `@preview/equator` — enter the picture, not as this package’s reason to exist, but as one further demonstration: that tools built independently on this backend, by people who’ve never seen each other’s code, compose without friction.

2. What else you could build

The examples ahead are one shape of a much larger family. Anything with the form “a source document, plus a companion that cites it with real page numbers, computed in the same pass” fits the same three-piece pattern: an anchor at each interesting location, a satellite that queries them, `bundle` to produce both files. None of the following exist as packages today — they’re sketches, meant to show the range this covers, not a roadmap:

- **A list of figures or tables**, generated instead of hand-maintained — immune to renumbering when a figure moves, impossible to forget to update.
- **A glossary or index of terms** — exactly the worked example in the next chapter, but just as at home in a legal contract’s list of defined terms or a textbook’s index.
- **An answer key**, kept separate from the exam it belongs to, citing the real question numbers and page each one landed on after layout.
- **An executive summary** that auto-cites the real page of whichever section of the full report it’s summarizing.
- **A preview/final split with nothing to do with revision tracking** — a form whose preview build shows field ids or validation state inline (via `preview()`), never present in the version actually handed to someone.
- **Supplementary materials** that cite “as shown in Figure 3, p. 7” of a main paper compiled in the very same pass, so the citation can never drift out of sync with a page a late edit renumbered.

Each is the same handful of primitives from the next two chapters, arranged differently — nothing app-specific ever needs to live inside `contexture` itself for any of them to work.

3. Installation and compiling

```
#import "@preview/contexture:0.1.0": *
```

A bundle compiles with the `bundle` feature and format flags Typst’s experimental export requires; everything else about **how many** files come out, and what they’re named, is decided by what’s listed under `documents`: (see “`satellite` and `bundle`” below), not by anything on the command line:

```
typst compile --features bundle --format bundle main.typ
```

Two flags, read by `variant()/preview()`, are available to every satellite regardless of which package defined it — see “Two independent compile axes” below for why there are two, not one:

```
typst compile --features bundle --format bundle --input variant=tracked main.typ
typst compile --features bundle --format bundle --input preview=true main.typ
```

4. The anchor primitive: `anchor`, `anchors`

`anchor(kind, payload)` marks the current location with a namespaced, queryable piece of data. `kind` namespaces it so two unrelated packages picking the same `id` scheme never collide — by convention, prefixed with the owning package’s name (“equator-item”, “palimpsest-passage”). `payload` is whatever that package needs back later — entirely opaque to `contexture` itself. Deliberately renders nothing on its own: emitting the metadata and deciding how (or whether) to render content around it are two different concerns, left to the caller.

`anchors(kind)` every anchor of that kind, in document order, from **anywhere in the bundle** — including a document other than the one this is called from, which is the entire point. Must be called from within a context.

This pair is the one mechanism every “cite this spot from another document, with its real page number” feature in this ecosystem is built from. The example below builds a small one from scratch: `term()` anchors a short definition where it’s first used in the manuscript; a `glossary` satellite lists every one of them with the real page it was found on. Neither function exists in `contexture` itself — this is exactly what a package built on it looks like, in miniature:

```
#import "../lib.typ": *

// A tiny satellite package, built from scratch out of contexture's
// primitives alone --- no palimpsest, no equator. `term()` anchors a
// short definition where it's first used; `render-glossary()` lists
// every one of them, in document order, with its real page number.

#let term(id, body) = {
  anchor("demo-term", (id: id, body: body))
  body
}

#let render-glossary() = context {
  let hits = anchors("demo-term")
  if hits.len() == 0 {
    [No terms defined.]
  } else {
    for h in hits [
      *#h.value.id* --- #h.value.body (p. #h.location().page())
      #linebreak()
    ]
  }
}

#let glossary = satellite("glossary", render: () => render-glossary())

#show: bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 10.5pt)
    body
  },
  documents: (glossary,),
)

= Introduction

The trial used #term("itt")[intention-to-treat] analysis throughout.

= Methods

Randomisation used #term("block")[permuted blocks] of size 4.
```

manuscript.pdf:

Introduction

The trial used intention-to-treat analysis throughout.

Methods

Randomisation used permuted blocks of size 4.

glossary.pdf, from the very same compile, citing the real pages the manuscript above was just laid out with:

itt — intention-to-treat (p. 1)
block — permuted blocks (p. 1)

4.1. Re-emitting stored content: strip-labels

A term's body can be arbitrary content, including a labelled figure — and re-emitting that figure verbatim into the glossary would otherwise plant a second copy of its label, which Typst rejects outright as a duplicate. `strip-labels(node)` reconstructs `node` with every label removed, and — specifically for a `figure`, a labelled block equation, or a heading — pins the **real**, already-resolved number of the true original onto the copy's own numbering, read directly via a fresh `query()` of that label. The copy shows the same number as the original; only the original stays a real, referenceable target.

```
#import "../lib.typ": *

// Same idea as bundle-glossary-basics.typ, but one term's body is itself
// a labelled figure --- re-emitting it verbatim in the glossary would
// otherwise plant a second copy of <tab-doses>, which Typst rejects as a
// duplicate label. strip-labels() drops the label from the copy while
// pinning the real, original "Figure 1" number onto it, read directly
// off the true instance via a query --- so both copies show the same
// number, and only one of them is a real, referenceable target.

#let term(id, body) = {
```

```

    anchor("demo-term", (id: id, body: body))
  body
}

#let render-glossary() = context {
  for h in anchors("demo-term") [
    *#h.value.id* (p. #h.location().page()):
    #strip-labels(h.value.body)
    #v(0.5em)
  ]
}

#let glossary = satellite("glossary", render: () => render-glossary())

#show: bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 10.5pt)
    body
  },
  documents: (glossary,),
)

= Methods

#term("doses")[
  #figure(
    table(columns: 2, [*Arm*], [*Dose*], [A], [10mg], [B], [Placebo]),
    caption: [Study drug doses.],
  ) <tab-doses>
]

As shown in @tab-doses, two arms were compared.

```

Both the manuscript and the glossary show “Table 1” — the genuine, resolved number, not a guess:

Methods

Arm	Dose
A	10mg
B	Placebo

Table 1: Study drug doses.

As shown in Table 1, two arms were compared.

manuscript.pdf

doses (p. 1):

Arm	Dose
A	10mg
B	Placebo

Table 1: Study drug doses.

glossary.pdf

`collect-metadata(body, tag)` and `collect-labels(body)` are the structural building blocks `strip-labels` and `anchor/anchors` themselves are built from — a purely structural walk of an in-memory content tree, no context

or layout involved, exposed directly for a package that needs to inspect content it's holding but hasn't (or may never) place into any document this compile. `is-blank(body)` and `is-textual(body)` answer two narrower questions the same way: whether `body` contains any real text at all (used by both `palimpsest` and `equator` to flag an accidentally empty marking call), and whether `body` is safe to wrap in literal quotation marks without producing a stray quote mark floating above a figure or table.

4.2. A fuller example: list of figures

The glossary above makes the mechanism easy to follow, at the cost of being a little toy — two terms, one line each. Here's a version closer to what you'd actually paste into a real project: a "List of Figures" companion, several entries deep, spanning a page break, each citing the real page its figure landed on.

```
#import "../lib.typ": *

// A fuller example than the glossary above -- closer to something you
// might actually copy into a real project: a "List of Figures"
// companion, generated instead of hand-maintained, citing the real page
// each figure landed on. Numbers figures in anchor order (1, 2, 3, ...)
// rather than reading Typst's own figure counter -- simpler, and exact
// here since fig() is the only thing creating figures in this document;
// a project mixing fig() with bare figure() calls would want
// counter(figure).at(..) instead.

#let fig(caption, body) = {
  let content = figure(body, caption: caption)
  anchor("demo-figure", (caption: caption))
  content
}

#let render-list-of-figures() = context {
  let hits = anchors("demo-figure")
  for (i, h) in hits.enumerate() [
    *Figure #(i + 1).* #h.value.caption --- p. #h.location().page()
    #linebreak()
  ]
}

#let list-of-figures = satellite("list-of-figures", render: () => render-list-of-figures())

#show: bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 10.5pt)
    body
  },
  documents: (list-of-figures,),
)

= Methods

#fig([Study flow diagram.], rect(width: 3cm, height: 1.5cm, align(center + horizon)[Flow]))

#lorem(90)
#pagebreak()

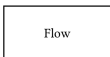
= Results

#fig([Primary outcome over time.], rect(width: 3cm, height: 1.5cm, align(center + horizon)[Chart A]))

#fig([Subgroup analysis.], rect(width: 3cm, height: 1.5cm, align(center + horizon)[Chart B]))
```

manuscript.pdf — three ordinary figures across two pages, nothing about them different from any other Typst document:

Methods



Flow

Figure 1: Study flow diagram.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam ipsum accusantium odio delectus est. Ut enim ad voluptatem, cum corpore doleamus, fieri tamen permagna accessio potest, si aliquid aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis.

Flow

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam ipsum vehicula sed magna ut aliquam erat volutpat. Ut enim ea doleamus animo, cum corpore dolemus, fieri tamen per magna accessio potest, si aliquid aeternum et infinitum impendere malum nobis opigeri. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augere amplificari non possit. At etiam Athenis, ut e patre audiam facete et urbane Stoicos irridere, statua est in qua a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possumus, omnis.

Results

A rectangular box with a black border, containing the text "Chart A" in the center.

Figure 2: Primary outcome over time.

A rectangular box with a black border, containing the text "Chart B" in the center.

Figure 3: Subgroup analysis.

Chart A

Chart B

page 2

list-of-figures.pdf, from the same compile:

Figure 3. Subgroup analysis. — p. 2

`fig()` here numbers its own entries by the order `anchor()` calls arrive in, rather than reading Typst's built-in `counter(figure)` — simpler, and exactly right as long as `fig()` is the only thing creating figures in the document (true here; a project mixing `fig()` with bare `figure()` calls would want `counter(figure).at(hit.location())` instead, the same real-counter trick `strip-labels` already uses above). Either way, the page numbers are never guessed or hand-typed — they come from `location().page()` on the anchor Typst itself placed, in the very compile that produced `manuscript.pdf`.

5. `satellite` and `bundle`: the shared pilot

`document(...)` — Typst’s own primitive for naming one output of a bundle compile — cannot be nested inside another `document(...)` call. That means two packages each shipping their **own** pilot (each calling `document(...)` internally, each convinced it alone owns the split between the manuscript and everything else) can never be stacked in the same compile. `contexture.bundle` is the fix: the **only** place, in this entire ecosystem, that ever calls `document(...)`. Every package built on `contexture` instead exposes a small constructor that returns a `satellite(...)` value — inert data, not a `document(...)` call — and the author lists as many of those as they like under one shared `documents::`.

`satellite(name, render:, applicable:, side-content: none)` describes one document to build alongside the manuscript. `name` is the base filename (`bundle` appends the variant/preview suffix). `render() -> content` produces this document’s content, called only when `applicable() -> bool` (default: always) says yes for the current compile — neither takes `variant/preview` as parameters: they’re plain global functions, so a `satellite` that cares calls `variant()/preview()` itself, right inside its own closure, rather than having them threaded in (equator’s checklist does exactly this in its own `applicable`, below).

`bundle(template:, documents: (), strict: false, manuscript-name: "manuscript", body)` the pilot itself, called via `#show: bundle.with(...)` — `body` is the rest of the document (typically `#include "manuscript.typ"`). Builds the manuscript (`template(body)`) plus every `satellite` whose `applicable` returns true, each as its own real Typst document sharing this one compile’s introspection space with all the others.

From `bundle-glossary-basics.typ` above, the wiring is just:

```
#let glossary = satellite("glossary", render: () => render-glossary())

#show: bundle.with(
  documents: (glossary,),
)
```

Restricting a compile to fewer documents: `--input only=<comma-separated satellite names>` restricts a single compile to the manuscript plus just the named satellites — `--input only=` with nothing after it produces the manuscript alone. A command-line choice for a fast preview, not a property of the project: naming a `satellite` under `only:` that declines to build itself this compile (its own `applicable` says no) doesn’t force it to.

`side-content:` content a `satellite` wants placed in the manuscript regardless of whether **it itself** gets built this compile. Exists for one confirmed need so far: a `satellite` whose own anchors need to be registered even when a fast preview skips it (`--input only=manuscript`) — otherwise some other check elsewhere in the manuscript (e.g. “this anchor has no matching response”) would false-positive on every single anchor, purely because the `satellite` that would have answered them wasn’t built this time. Paired with `collect-anchors(body, kind)` — the structural counterpart to `anchors`, finding anchors already sitting inside an in-memory `body` that was never placed into any document — and `reemit(collected)`, which registers one such found anchor as if it had been placed here.

6. Two independent compile axes: `variant`, `preview`

`variant()` "plain" (default) or "tracked", from `--input variant=...` An **intrinsic** property of what the manuscript’s own content means — do revision marks show, styled, or not. A package with no such concept (a reporting-checklist package, say) never touches this and always sees "plain".

`preview()` true/false, from `--input preview=true`. Whether to show debug highlighting for wherever a package’s own anchors sit in the manuscript — a drafting aid, never present in a real deliverable, and never a reason to change what a document’s content **means** (that’s `variant`’s job).

These are deliberately two separate flags, not one shared “mode” string, because of a concrete bug found combining two real packages built independently on the same idea: each treated “anything other than the default” as “my own alternate mode is on,” so a preview request meant for one package silently flipped the other’s rendering too, because both happened to read the same flag under incompatible vocabularies (`clean/tracked` vs `clean/annotated`, in the flag’s very first version). Splitting the concept into two flags with two clearly distinct meanings, each with its own boolean-or-enum shape, made that collision structurally impossible rather

than merely documented against. (`variant()`’s own default sentinel was renamed once more since, from "clean" to today’s "plain" — "clean" was itself borrowed from one specific consumer’s vocabulary, not a neutral word; see `MULTI-DOCUMENT-BUNDLE-DESIGN.md` for the full story.)

Two independent, made-up satellites below — neither is palimpsest’s or equator’s — each read the two axes directly, to show they’re plain, package-agnostic values, and that they never leak into each other:

```
#import "../lib.typ": *

// Two independent, made-up satellites, each reading the two shared
// compile axes directly -- neither is palimpsest's or equator's, to show
// that `variant`/`preview` are plain, package-agnostic flags any
// satellite can hook into, and that they never leak into each other.

#let note = satellite(
  "note",
  applicable: () => variant() == "plain" and not preview(),
  render: () => [This satellite only builds under variant=plain, preview=false.],
)

#let tracked-only = satellite(
  "tracked-only",
  applicable: () => variant() == "tracked",
  render: () => [This satellite only builds under variant=tracked.],
)

#show: bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 10.5pt)
    body
  },
  documents: (note, tracked-only),
)

= Manuscript

#context [Variant: #variant() --- Preview: #preview()]
```

Compiling this one file four ways produces four genuinely different combinations, each satellite gated independently:

Compile	Files produced
(no flags)	manuscript.pdf, note.pdf
--input variant=tracked	manuscript-tracked.pdf, tracked-only-tracked.pdf
--input preview=true	manuscript-preview.pdf
--input variant=tracked --input preview=true	manuscript-tracked-preview.pdf, tracked-only-tracked-preview.pdf

`note` builds exactly when `variant() == "plain"` and `preview()` is off; `tracked-only` builds exactly when `variant() == "tracked"`, regardless of `preview()`. Turning `preview` on never affects whether `tracked-only` builds, and switching `variant` never affects `note`’s own `preview`-gating — the two axes compose freely because neither satellite (nor `bundle` itself) ever collapses them into one shared value.

`bundle` folds both axes into the manuscript’s own filename independently, in that same orthogonal way — `manuscript.pdf` / `manuscript-tracked.pdf` / `manuscript-preview.pdf` / `manuscript-tracked-preview.pdf` — so a `preview` compile never silently overwrites the plain deliverable, whatever `variant` happens to be at the same time, and a `tracked-and-preview` compile is simply both suffixes, in order.

7. Diagnostics: diagnose, set-strict

Typst has no public API to emit a soft compiler warning from user code, so the closest available approximation is a visible marker rendered directly at the fault location, which most Typst editors preview live.

`diagnose(message, always: false)` reports a problem at the call site. Under `set-strict(true)`, always a hard panic — a real compile error, in any mode. Otherwise, a visible inline marker — muted specifically when

`variant() == "plain"` and `preview()` is off (always: false, the default), since that's the file most likely to leave this codebase and reach someone who never asked to see an internal warning; shown unconditionally when `always: true`, for a diagnostic embedded in a document that is **never** itself the deliverable (a generated report, an internal checklist) where muting it would mean it's never seen at all.

`set-strict(v)` turns every `diagnose(...)` call, from any package built on `contexture`, in any document, into a hard error at once — one shared CI gate rather than one per package. Normally set via `bundle(strict: true, ...)`, not called directly.

`xref` (below) always passes `always: true` — a broken cross-reference should never be silently invisible even in the real, submitted deliverable:

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set heading(numbering: "1.")

= Results <sec-results>

#figure(table(columns: 2, [*Arm*], [*n*], [A], [120], [B], [118]), caption: [Participants per
arm.]) <tab-n>

See #xref(<tab-n>) and #xref(<sec-results>) above. This one doesn't exist: #xref(<tab-missing>).
```

1. Results

Arm	n
A	120
B	118

Table 1: Participants per arm.

See Table 1, p. 1 and Section 1, p. 1 above. This one doesn't exist: ⚠ xref(<tab-missing>): not found.

8. xref: cross-references with a real page number

`xref(label)` behaves like `@label/ref(label)` — which already resolves across documents in a bundle, a satellite's own `@tab-results` renders the manuscript's real "Table 3" — but appends the real page number: "Table 3, p. 14". Explicit rather than a bare `@label`, so it stays correct even if a future document duplicates the same label, where a bare `ref` would become ambiguous between the two copies. It operates on plain Typst labels (figures, headings, equations, ...), not on `contexture.anchor()` — a separate, narrower tool from the anchor primitive above, for the common case where a real Typst label already exists and only the page number needs adding.

9. Composing independent packages

Two packages built independently on `contexture`, neither aware the other exists, combine by nothing more than listing both of their satellites under the same documents: — the scenario the whole design exists for. `@preview/palimpsest`'s `letter(...)` and `@preview/equator`'s `checklist(...)` are each just a `satellite(...)` value; `bundle(...)` (this package, not either of theirs) is still the only thing that ever calls `document(...)`.

```
#import "../../../typst-palimpsest/lib.typ" as palimpsest
#import "../../../typst-equator/lib.typ" as equator
#import "../lib.typ": bundle

// The flagship "composition" example: two independently-developed
// packages, each built on contexture and neither aware the other
// exists, combined in one compile through nothing more than one shared
// `documents:` list. `letter(...)` and `checklist(...)` are each just a
// `satellite(...)` value; `bundle(...)` (this package, not either of
// theirs) is the only thing that ever calls `document(...)`.
```

```

#let tiny-checklist = (
  name: "TINY",
  full-name: [Tiny reporting checklist],
  items: (
    (section: "Methods", topic: "Randomisation", group: none, id: "1",
      description: [How the allocation sequence was generated.]),
    (section: "Methods", topic: "Outcome assessment", group: none, id: "2",
      description: [How the primary outcome was assessed.]),
  ),
)

#let my-template(body) = {
  set page(width: 16.6cm, height: auto, margin: 12pt)
  set text(size: 10.5pt)
  body
}

#let exchanges = palimpsest.reviewer(1)[
  #palimpsest.exchange(<r1-1>)[Please clarify whether outcome assessment was blinded.][
    Blinding is now specified. #palimpsest.pinpoint(<r1-1>)
  ]
]

#show: bundle.with(
  template: my-template,
  documents: (
    palimpsest.letter(exchanges: exchanges),
    equator.checklist(checklist: tiny-checklist),
  ),
)

// Item 1 has nothing to do with this reviewer exchange -- check() just
// renders its own text at its own spot, independently of passage().
#equator.check("1")[Randomisation used a computer-generated sequence.]

// Item 2, though, IS the reviewer exchange: the added text is both a
// tracked revision AND the manuscript's answer to item 2. check(id, body)
// and passage() would each render their own body -- used together on
// the exact same span, that would print it twice. check(id)'s bare,
// point-marker form registers item 2's coverage without rendering
// anything; passage() is the only call that actually prints the text,
// once, and handles its tracked-mode marks.
#palimpsest.passage(<r1-1>)[
  The primary outcome was assessed #palimpsest.add[by a rater blinded to group assignment].
]
#equator.check("2")

```

One compile, four ways — each producing a genuinely different combination of files, exactly like the made-up example in “Two independent compile axes” above, now with two real packages instead of a toy demonstration:

Compile	Files produced
(no flags)	manuscript.pdf, response.pdf, checklist.pdf
--input variant=tracked	manuscript-tracked.pdf, response-tracked.pdf
--input preview=true	manuscript-preview.pdf, response-preview.pdf
--input variant=tracked --input preview=true	manuscript-tracked-preview.pdf, response-tracked-preview.pdf

checklist.pdf only ever comes out of the first, plain compile — checklist(...)’s own applicable rule declines under either a tracked variant or preview mode, since either could shift page breaks relative to the real, submitted manuscript (see equator’s manual for the full reasoning). Palimpsest’s letter(...) has no such restriction — a response letter is meaningful, and useful to preview, in every combination.

The last (tracked **and** preview) compile shows both packages’ overlays together, in the same document, each independent of the other — item 1 gets equator’s full preview highlight box (its own span has nothing to do with the reviewer exchange), item 2 gets palimpsest’s tracked-mode revision mark **and** a small [2] tag from equator’s own bare, point-marker form (see below for why it’s a tag with no box, rather than a full highlight):

Randomisation used a computer-generated sequence. ^[1]

The primary outcome was assessed [by a rater blinded to group assignment](#). ^{[R1-1] [2]}

Rule 1 — don’t nest one package’s marking function inside another’s. `#check(...)[#passage(...)[...]]` and the reverse each break something, for the same underlying reason both times: `passage()`’s own visual rendering and `check()`’s own preview-mode highlighting are each wrapped in a context block (needed to read live style state), and a context block is structurally opaque to anything trying to inspect its contents **before** layout. Nest `passage()` inside `check()` and `check()`’s own blank-content self-check can no longer see the real text inside — it misreports the passage as empty, in **any** mode, not just under `preview: true`. Nest `check()` inside `passage()` and, under `preview: true` specifically, `passage()`’s own scan for `add/del/rep` marks can no longer see them — it misreports “contains no mark”. Two different symptoms, one cause, no nesting order avoids it.

Rule 2 — don’t call them as two independent, rendering siblings on the exact same span either. This is item 1’s pattern above, and it’s correct **there** only because item 1’s span and the reviewer exchange are two different things. Try it on the **same** span — the reviewer’s requested change genuinely is the manuscript’s answer to a checklist item, item 2’s case — and both calls render their own body, so the text prints twice, plainly, visibly duplicated:

```
// DON'T -- prints the sentence twice when body is the same text:
#check("2") [The primary outcome was assessed by a rater blinded to group assignment.]
#passage(<r1-1>)[
  The primary outcome was assessed #add[by a rater blinded to group assignment].
]
```

This is exactly why `check` (like palimpsest’s own `passage`) accepts two shapes rather than one: `check(id, body)` for the common case, and a bare `check(id)` — no second argument at all — for exactly this collision. The bare form registers the same metadata (same page resolution, same everything `checklist.pdf` needs) but renders nothing beyond that small `[2]` tag under `preview: true`, and nothing at all in the plain compile — there’s no body here to duplicate or to nest, which is the whole point:

```
// item 2's actual code, above -- one render, one registration:
#passage(<r1-1>)[
  The primary outcome was assessed #add[by a rater blinded to group assignment].
]
#check("2")
```

Neither rule is specific to palimpsest or equator. Both follow directly from anchor’s own design: an anchor’s content is rendered by whatever wraps it, structurally, so a structural pre-layout scan can only see through wrappers it already knows about (rule 1), and two independent calls that each render the same content will always render it twice, since nothing about `bundle`, `satellite`, or `anchor` deduplicates rendered output across calls (rule 2). Any future package built on `contexture` whose marking function renders its own body should expose the same two shapes — a full form for the common case, and a bare, render-free form for when its own span coincides with something another package already renders (the way palimpsest’s own `passage(anchors, body)/passage(body)` already does, for an unrelated reason — arity dispatch turns out to be a natural fit for “this call sometimes needs less than its full argument list” in general, not just for this one case). Palimpsest’s own manual walks through rule 1 in more depth, including the two failure modes as they actually looked before the fix; equator’s manual covers `check(id)`’s own signature and trade-offs.